

NetZIP: Algorithm/Hardware Co-design of In-network Lossless Compression for Distributed Large Model Training

Jinghan Huang
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
jinghan4@illinois.edu

Hyungyo Kim
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
hyungyo2@illinois.edu

Nachuan Wang
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
nachuan3@illinois.edu

Jaeyoung Kang
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
jyakang5@illinois.edu

Hrishi Shah
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
hrishis2@illinois.edu

Eun Kyung Lee
IBM Research
Yorktown Heights, New York, USA
eunkyoung.lee@us.ibm.com

Minjia Zhang
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
minjiaz@illinois.edu

Fan Lai
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
fanlai@illinois.edu

Nam Sung Kim
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
nam.sung.kim@gmail.com

Abstract

In distributed large model training, the long communication time required to exchange large volumes of gradients and activations among GPUs dominates the training time. To reduce the communication times, lossy or lossless compression of gradients and/or activations can be employed. However, lossy compression of gradients and activations may demand more training iterations to achieve the same model accuracy and cause convergence failure, respectively. Lossless compression, on the other hand, may not reduce the volumes of gradients and activations enough to offset the significant latency associated with compression and decompression on current platforms. To address these challenges, we propose NETZIP, an algorithm/hardware co-design for in-network *lossless* compression of both gradients and activations. NETZIP consists of two components. (1) NETZIP-algorithm transforms gradients and activations at the bit and value levels to help lightweight standard lossless compression achieve more compression of the gradients and activations. (2) NETZIP-accelerator integrates NETZIP-algorithm with a lightweight lossless compression accelerator within a NIC in a bump-in-the-wire fashion to reduce the compression/decompression latency under the resource constraints. NETZIP-algorithm compresses gradients and activations 40–63 and 43–75 percentage points more, respectively, than heavy standard lossless compression for Llama-3 70B, GPT-3 175B, and Llama-3 405B. NETZIP-accelerator, implemented within FPGA-NICs and connected to commodity servers, provides orders of magnitude lower

latency for compression and decompression, respectively, than the lowest latency achieved by the standard lossless compression on current platforms. With greater compression of both gradients and activations, and lower latency for compression and decompression than the standard lossless compression, NETZIP provides 35% lower training time.

CCS Concepts

• **Computer systems organization** → **Neural networks**; • **Networks** → **In-network processing**.

Keywords

Foundation Model, Distributed Training, Network, Compression

ACM Reference Format:

Jinghan Huang, Hyungyo Kim, Nachuan Wang, Jaeyoung Kang, Hrishi Shah, Eun Kyung Lee, Minjia Zhang, Fan Lai, and Nam Sung Kim. 2025. NetZIP: Algorithm/Hardware Co-design of In-network Lossless Compression for Distributed Large Model Training. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3725843.3756079>

1 Introduction

Foundation models, such as GPT [50] and Llama [15], have revolutionized the field of machine learning (ML), particularly in natural language processing (NLP) and computer vision. These powerful, general-purpose models capture a broad range of knowledge and understanding, but the size of these models and the volume of training data required are massive [37]. Therefore, a given model and its training data need to be distributed across clusters of high-end compute nodes, such as those based on NVIDIA DGX [44], to be trained within a tractable amount of time [13, 49]. Each such node—comprising multiple GPUs connected by high-bandwidth



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/25/10

<https://doi.org/10.1145/3725843.3756079>

(intra-node) interconnects, such as NVLink [47]—is connected to other nodes through multiple high-bandwidth (inter-node) network links, such as 400 Gbps Ethernet [46].

As organizations or individuals with limited resources cannot afford to use such expensive clusters to develop their own models, they leverage these foundation models as starting points and fine-tune them for their specific applications using public clouds providing clusters of more affordable compute nodes [60]. Each such node is connected to a modest-bandwidth network link, such as 50 or 100 Gbps Ethernet at best [8]. As it consists of one or two GPUs at most, it is not equipped with a high-bandwidth intra-node interconnect, which further increases the volumes of gradients and activations exchanged across nodes through its network link, compared to a high-end compute node that exchanges a large fraction of gradients and activations within the node. Meanwhile, the training time is already dominated by the communication time required to exchange gradients and activations across compute nodes (referred to as inter-node communication time henceforth), even on clusters of high-end compute nodes [27, 61]. Consequently, training large models on clusters of affordable compute nodes inevitably and considerably increases the inter-node communication time compared to clusters of high-end compute nodes.

To reduce the inter-node communication time, one of the most cost-effective solutions is to employ compression within distributed training frameworks, reducing the volumes of gradients and activations before exchanging them across compute nodes. Toward this goal, lossy compression has been popular and widely explored, but it suffers from two drawbacks [65]. First, training with lossy compression of gradients often demands more training iterations to achieve the same model accuracy as training without compression [16]. Second, training with lossy compression of activations, which we find contributes to a large percentage of the inter-node communication time during large-model training, may cause sub-optimal convergence or even convergence failure [53, 55]. Alternatively, lossless compression can be applied to both gradients and activations without suffering from the two drawbacks of lossy compression. However, we show that lossless compression of gradients and activations in bfloat16 format fails to sufficiently reduce their volumes to offset the significant latency associated with compression and decompression on current platforms, including the CPU, GPU, and SmartNIC (SNIC).

To address these challenges, we propose NETZIP, an algorithm/hardware co-design for in-network *lossless* compression of both gradients and activations with the following specific contributions.

Impact of employing lossless compression on the end-to-end inter-node communication time on current platforms.

We first evaluate four popular lossless compression algorithms—LZ4 [33], Snappy [14], Zstd [11], and Deflate [34]—by measuring their compression ratios¹ for gradients and activations of Llama-3 70B, GPT-3 175B, Llama-3 405B. This shows that lightweight algorithms—LZ4 and Snappy—cannot compress both gradients and activations at all, while heavy algorithms—Zstd and Deflate—can compress them only modestly (19–28% reduction in their volumes).

Second, we implement these algorithms within a distributed training framework, allowing each to compress/decompress gradients and activations on the current platforms—CPU, GPU, and SNIC—during model training. The evaluation shows that training with Deflate, for example, on the CPU, GPU, and SNIC increases the end-to-end inter-node communication time by 8.8×, 1.8×, and 3.7×, respectively, compared to training without compression. This is because the increase in communication time associated with compression and decompression is notably longer than the decrease in communication time achieved by sending compressed gradients and activations.

Lossless compression enhancement algorithms. Our analysis of gradients and activations at the bit and value levels shows that their exponent bits are more skewed toward 1s (*i.e.*, lower entropy or more compressible), while their mantissa bits are random, with narrow distributions of gradient and activation values close to zero. However, LZ4 and Snappy, which treat a byte as a character and perform dictionary matching with a sliding window, cannot identify repeated character strings due to random mantissa bits in every other byte. Thus, we propose grouping bytes or bits across all gradients or activations with the same byte or bit index within each bfloat16 value. This allows LZ4 and Snappy to compress them considerably more. Additionally, based on the intuition that both gradients and activations change gradually over training iterations, we propose subtracting, for example, a representative gradient value from the previous iteration from every gradient value at the current iteration. This makes their mantissa bits skewed toward 0s and thus more compressible. We refer to these two algorithms collectively as NETZIP-algorithm henceforth. On average across the three models, LZ4, for instance, enhanced by NETZIP-algorithm compresses gradients and activations 60 and 70 percentage points more than standard LZ4, respectively.

Bump-in-the-wire compression/decompression accelerator within a NIC. We observe that the additional transfer of gradients and activations, required for compression and decompression by the CPU, GPU, and SNIC, contributes significantly to the end-to-end compression/decompression latency. Besides, Zstd and Deflate provide more compression than LZ4 and Snappy, but at the expense of lower throughput, higher latency, and also greater hardware resource and power consumption, especially when implemented in hardware, which has limited the Deflate compression accelerator's throughput in commodity SNICs. Considering these, we propose NETZIP-accelerator, a bump-in-the-wire accelerator integrating NETZIP-algorithm with a lightweight LZ4 accelerator. As a bump-in-the-wire accelerator, it obviates the additional transfer of gradients and activations for compression and decompression. Using LZ4, it maximizes the throughput of compression and decompression while minimizing their latency under resource and power constraints. Overall, it can provide 1.7× and 2.1× higher throughput and 2.0× and 2.1× lower latency than Zstd and Deflate, respectively, when implemented in hardware under the same resource constraints. With more compression and lower compression/decompression latency together, training with NETZIP provides 35% lower training time, respectively, than training without compression.

¹A compression ratio is defined as the volume of compressed gradients or activations divided by that of uncompressed gradients or activations.

2 Background

2.1 Distributed Training Frameworks

Training a large model, such as Llama-3 405B, often requires tens of millions of iterations [15]. Each iteration includes a forward pass to compute prediction loss against labeled data, followed by a backward pass to adjust model weights based on gradients. The state-of-the-art distributed training frameworks such as Megatron-LM [54] and DeepSpeed [52] have primarily exploited three types of Parallelism to scale model training across many compute nodes.

(P1) Data parallelism (DP) shards a given training dataset into multiple subsets and distributes them across GPUs, each training a copy of a given model. It requires global synchronization of all gradients after every backward pass in each iteration to make sure that every copy across all GPUs has the same set of parameters. **(P2) Pipeline parallelism (PP)** breaks a given model into multiple stages, each consisting of consecutive layers, and assigns each stage to a GPU, requiring a transfer of activations from one GPU to another. **(P3) Tensor parallelism (TP)** further divides a layer's weights, thus computations, across GPUs to reduce memory requirements for large models. The activations of these sub-layers are synchronized across GPUs to complete the per-layer computations.

While each type of parallelism presents distinct trade-offs among computation, communication, and memory costs, popular distributed training frameworks often use all three types of parallelism to maximize training efficiency. With the skyrocketing increase in model sizes, distributed training exchanges rapidly growing volumes of gradients and activations across GPUs, placing increasing demands on the network infrastructure to provide high bandwidth [35].

2.2 Networking Infrastructure for Distributed Large Model Training

The network infrastructure for distributed large model training varies widely. To quickly exchange large volumes of computed gradients and activations (collectively referred to as intermediate values) across hundreds to thousands of GPUs, top-tier hyperscale clusters connect each of their compute nodes to multiple 400 Gbps network links [13, 49]. Nonetheless, affordable clusters in public clouds still depend on modest-to-low bandwidth network links. For example, CloudLab clusters [9] provide a convenient environment for model training at little to no cost, but the bandwidth of their

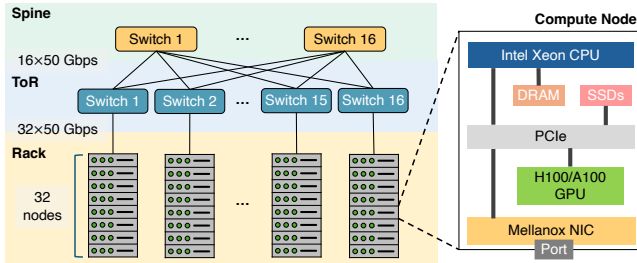


Figure 1: Common datacenter network topology with a zoomed-in view of a compute node based on a CloudLab instance.

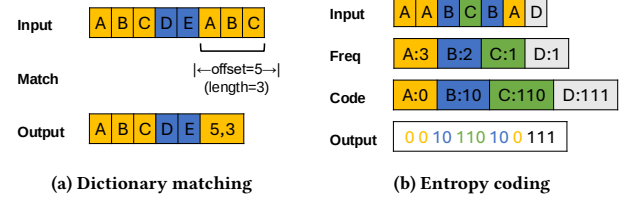


Figure 2: Representative lossless compression algorithms.

network links typically ranges from as low as 10 Gbps to commonly 50 Gbps [26, 51]. Figure 1 depicts such a cluster with a common datacenter network topology (also referred to as DCN+ [49]) and a compute node architecture based on a high-end CloudLab instance (nvidiagh [8]). The network topology is a hierarchical, three-tier Clos network, with top-of-rack (ToR) and spine switches constituting a non-blocking fabric. Each node consists of a server-class CPU with either a past or current generation of GPU (e.g., an Intel Xeon CPU with an NVIDIA A100 or H100 GPU) and a Mellanox ConnectX-5 50 Gbps NIC.

The distributed training frameworks (§2.1) generate highly concentrated bursts of large network packets to exchange gradients and activations [13]. Such network traffic patterns place significant pressure on the network infrastructure, causing severe congestion and, consequently, a substantial increase in the time required to exchange gradients and activations across GPUs. This is particularly problematic when the network bandwidth is limited, as is often the case with affordable clusters in public clouds. Consequently, distributed training time increases significantly, as GPUs cannot proceed with computing subsequent stages or layers until they receive these gradients and activations.

2.3 Lossless Compression Algorithms

Lossless compression algorithms are often based on dictionary matching (e.g., LZ77 [68]) and/or entropy coding (e.g., Huffman coding [18]) illustrated in Figure 2. Several lossless compression algorithms have been developed to balance among compression throughput, latency, and ratio. For example, LZ4 [33] and Snappy [14] employ dictionary matching. They use a sliding window to detect and encode repeated character strings, replacing subsequent occurrences of the character strings with a reference to their earlier position in the window. This approach achieves high compression/decompression throughput but delivers a relatively modest compression ratio. In contrast, Deflate [34] and Zstd [11] combine both dictionary matching and entropy coding. After locating repeating character strings with LZ77, Huffman coding assigns shorter bit sequences to more frequent characters.

3 Impact of Limited Network Bandwidth on Distributed Large Model Training

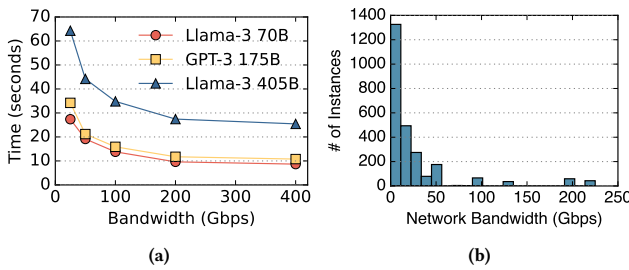
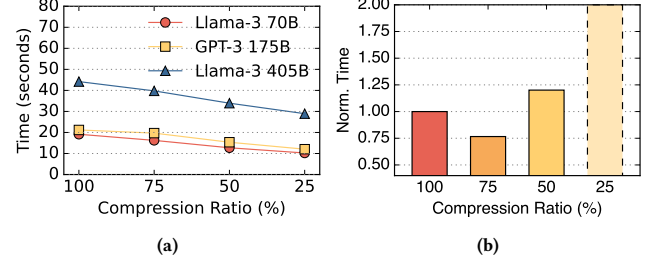
Setup. To evaluate the impact of limited network bandwidth on distributed large model training with affordable clusters in public clouds and for reproducibility (§2.2), we use SimAI [63], a full-stack, high-precision simulator developed by the industry. SimAI is

Table 1: Model configurations for SimAI. WS denotes the world size, i.e., the number of GPUs used.

Model	Parameters	Layers	Heads	TP	DP	PP
GPT-3	175B	96	96	8	WS/(PP × TP)	8
Llama-3	70B	82	64	8	WS/(PP × TP)	4
	405B	128	128	8	WS/(PP × TP)	16

capable of comprehensively simulating the entire lifecycle of distributed model training at a large scale, modeling not only collective communication primitives like AllReduce and ReduceScatter but also low-level network behaviors, such as congestion at network switches and NICs. We configure it to simulate clusters of 512 nodes, each comprising one NVIDIA H100 GPU, with the network topology depicted in Figure 1. Table 1 describes three models evaluated by SimAI—Llama-3 70B, GPT-3 175B, and Llama-3 405B—along with their distributed training strategies and numbers of parameters, layers, and attention heads. For efficient training across multiple GPUs, the models utilize all three types of parallelism: TP, DP, and PP (§2.1). DP is computed by $WS/(PP \times TP)$, where WS denotes the world size, i.e., the total number of GPUs in use. These models are trained using the Megatron-LM framework.

Communication Time vs. Network Bandwidth. Using the above setup, we first investigate the impact of network bandwidth on the inter-node communication time per training iteration while varying the network link bandwidth and proportionally scaling the network switch bandwidth, collectively referred to as the network bandwidth hereafter. Figure 3(a) shows that the communication time per training iteration begins to increase superlinearly as the network bandwidth decreases below 200 Gbps for Llama-3 70B, GPT-3 175B, and Llama-3 405B. Meanwhile, Figure 3(b) shows that 2,350 out of a total of 2,561 instances in a public cloud have a network bandwidth of 50 Gbps or less. These instances are connected via relatively modest-bandwidth network links in CloudLab [8]. Additionally, around 800 out of 850 instance types in Amazon EC2 have network bandwidths lower than 50 Gbps [3]. That is, instances with limited network bandwidth are common in public clouds, and not everyone can easily afford access to high-end instances with network bandwidth exceeding 100 Gbps. Based on this observation, when the network bandwidth is decreased to 50 Gbps, the communication time per training iteration for Llama-3 70B, GPT-3 175B, and Llama-3 405B increases by 2.2×, 2.0×, and 1.7×, respectively,

**Figure 3: (a) Communication time per training iteration with varying the network bandwidth and (b) distribution of the NIC bandwidth across instances in a public cloud [8].****Figure 4: (a) Communication time per training iteration and (b) total training time, normalized to that without compression, with varying the compression ratio. The dashed bar indicates convergence failure.**

compared to a network bandwidth of 400 Gbps. Note that the overall time per training iteration is dominated by the communication time, and the communication time stops decreasing beyond 200 Gbps for Llama-3 70B and GPT-3 175B, and 400 Gbps for Llama-3 405B. This is due to the fixed latency of gradients and activations exchanged through network links and switches, which is determined by the network topology for a given number of nodes.

Total Training Time vs. Lossy Compression. To reduce the communication time under the limited network bandwidth, we may compress gradients and activations exchanged across nodes. Figure 4(a) shows that the communication time decreases as the compression ratio (or the volume of gradients and activations) decreases, assuming a network bandwidth of 50 Gbps. For instance, when the compression ratio decreases to 75%, the communication time of Llama-3 8B per training iteration decreases by 23%. However, applying the latest lossy compression technique—top-k gradient sparsification [55]—to gradients, we observe that the total training time of Llama-3 8B increases when the compression ratio falls below 50%. This is because training with such lossy compression increases the number of iterations required to accomplish the same model accuracy as training without compression, due to the accuracy loss incurred in each iteration, as also supported by recent work [10, 20, 57, 71].

Takeaway-1: Prior work employing lossy compression can considerably reduce the communication time per training iteration, but it may increase the total training time.

Distributed large model training with hundreds of nodes often exploits all three types of parallelism—DP, TP, and PP—requiring the exchange of not only gradients but also activations. We observe that the percentage of communication time contributed by exchanging activations is 27%, 21%, and 49% for Llama-3 70B, GPT-3 175B, and Llama-3 405B, respectively. Despite the high percentage of communication time contributed by exchanging activations, neither past work [28] nor the latest work [55] has successfully applied lossy compression to activations. This is because even a slight loss of accuracy in activations gives rise to a detrimental effect on model convergence, unlike gradients.

Takeaway-2: The exchange of activations also accounts for a large percentage of the communication time, but lossy compression cannot be applied to activations.

4 Limitations of Lossless Compression Using Commodity Computing Platforms

In this section, we first evaluate the compression ratios of four popular *lossless* compression algorithms, considering that *lossy* compression algorithms can increase the overall training time of models or cause them to fail to converge. We then measure the throughput and latency of compressing and decompressing intermediate values using these algorithms on three representative computing platforms: CPU, GPU, and SNIC.

Setup. To evaluate the throughput and latency of compression and decompression, we set up a server equipped with the 4th-generation Intel Xeon Scalable Processor, the NVIDIA H100 GPU, and the NVIDIA BlueField-2 SNIC (Section 6.1). We use the NVIDIA BlueField-2 SNIC since the latest NVIDIA BlueField-3 has only a decompression accelerator. Note that we use BlueField-2 specifically for analyzing the compression accelerator and node-to-node communication. However, our primary focus is on low-bandwidth NICs (e.g., 50 Gbps), as discussed and analyzed in Section 3. We start by collecting gradients and/or activations that each node sends to other nodes during each training iteration, based on DP, TP, and PP shown in Table 1. The CPU runs LZ4, Snappy, Zstd, and Deflate using standard C++ LZ4 [33], Snappy [14], Zstd [11], and Deflate [34], respectively, while the GPU runs them using the nvCOMP [45] library. The SNIC, which supports only Deflate with its hardware accelerator, runs Deflate using the DOCA Compress library [41]. For CPU-based compression and decompression, we divide the gradients and activations from fine-tuning iterations into 128 kB chunks and use all 40 CPU cores to compress each chunk and decompress it in parallel to maximize throughput. We also faithfully capture the end-to-end inter-node communication time, including compression, transfer through network links, and decompression of gradients and activations. For example, the CPU-based compression goes through the following path: sender GPU memory → CPU memory → CPU compression → CPU memory → NIC → receiver NIC → CPU memory → CPU decompression → CPU memory → GPU memory.

Compression Ratio. Table 2 presents the average compression ratios of the gradients and activations of Llama-3 70B, GPT-3 175B, and Llama-3 405B achieved by LZ4, Snappy, Zstd, and Deflate. This shows that LZ4 and Snappy, which rely on dictionary matching, cannot decrease the volume of either gradients or activations at all (i.e., a compression ratio of 100%) or even increase it due to the overhead of storing the dictionary. This is because LZ4 and Snappy use dictionary matching with a sliding window to find repeated 8-bit character strings, while the lower 8 bits of gradients and activations

in bfloat16 format (recognized as characters by LZ4 and Snappy) are mostly random, according to our analysis (§5.1), making it difficult to find repeated long 8-bit character strings. Meanwhile, Zstd and Deflate, which use both dictionary matching and Huffman coding, reduce the volumes of gradients and activations by 20%–28%, as Huffman coding can identify some common upper 8 bits across them. On average, Zstd provides 2–7% more compression for gradients and activations than Deflate, and it compresses activations 7% more than gradients. The reasons why Deflate and Zstd offer more compression than LZ4 and Snappy, as well as activations are more compressible than gradients, will be discussed later (Section 5.1).

Throughput and Latency. Table 3 provides the throughput and latency for compressing and decompressing intermediate values (i.e., gradients and activations) of Llama-3 70B, GPT-3 175B, and Llama-3 405B using the CPU, GPU, and SNIC. Since LZ4 and Snappy cannot compress gradients and activations at all, they will only increase the communication time between nodes due to the added compression/decompression latency, whether they run on the CPU or GPU. However, we evaluate their throughput and latency because they can highly compress gradients and activations with the assistance of NetZIP-algorithm (Section 5.1).

CPU-based LZ4 provides similar throughput compared to CPU-based Snappy, where ‘throughput’ refers to the average of compression/decompression throughput, unless explicitly specified. With simpler algorithms, CPU-based LZ4 offers 1.6–1.8× higher throughput than Zstd, and 6.6–11.0× higher throughput than CPU-based Deflate. GPU-based Snappy offers 1.1–6.1× higher throughput than LZ4 because it is designed to exploit parallelism better than LZ4, dividing given gradients or activations into smaller chunks (e.g., 32 kB) and compressing them in parallel. This higher throughput typically comes at the cost of less compression (i.e., a lower probability of finding repeated character strings within smaller chunks), although neither LZ4 nor Snappy compresses gradients and activations at all in this work. GPU-based Snappy offers 9.6–13.6× higher throughput than GPU-based Zstd, and 7.8–9.6× higher throughput than GPU-based Deflate. Lastly, SNIC-based Deflate provides 7–12% higher compression throughput than GPU-based Deflate, but it offers 25–88% lower decompression throughput.

Across all the lossless compression algorithms, the GPU, leveraging massive parallelism, can provide up to one order of magnitude higher throughput than the CPU for compression and decompression. GPU-based LZ4, Snappy, Zstd, and Deflate offer 2.9×, 8.3×, 1.2×, and 7.8× higher throughput than CPU-based LZ4, Snappy, Zstd, and Deflate, respectively. Nonetheless, GPU-based compression and decompression using nvCOMP have a drawback. For example, before the GPU sends gradients and/or activations to the NIC, it must read the uncompressed gradients and/or activations from GPU memory and then write the compressed gradients and/or activations back to GPU memory before the GPU sends them to the NIC. In such a case, without sophisticated optimization, the GPU may require GPU memory to store uncompressed/compressed gradients and/or activations at the same time, reducing the available memory capacity for storing tensors and/or layers that a single GPU can handle. Under the memory capacity constraint of a single GPU, GPU-based compression potentially demands even more GPUs for distributed large model training.

Table 2: Average compression ratios of gradients (G) and activations (A), along with their respective percentages in the total volumes of gradients and activations exchanged between nodes (in parentheses).

	Llama-3 70B		GPT-3 175B		Llama-3 405B	
	G (73%)	A (27%)	G (79%)	A (21%)	G (51%)	A (49%)
LZ4	102%	101%	101%	101%	101%	101%
Snappy	102%	102%	101%	101%	100%	100%
Zstd	80%	74%	79%	72%	80%	78%
Deflate	81%	80%	81%	79%	81%	80%

Table 3: Throughput and latency of lossless compression algorithms running on the CPU, GPU, and SNIC.

Algorithm	Platform	Throughput (Gbps) (Compression, Decompression)			Latency (ms) Total (Compression, Network Link, Decompression)		
		Llama-3 70B	GPT-3 175B	Llama-3 405B	Llama-3 70B	GPT-3 175B	Llama-3 405B
LZ4	CPU	(24, 130)	(42, 223)	(50, 243)	6.5 (3.4, 2.2, 0.9)	40.4 (16.0, 20.0, 4.4)	165.6 (62.8, 84.2, 18.6)
	GPU	(62, 420)	(66, 1212)	(63, 1639)	3.6 (1.2, 2.2, 0.2)	29.5 (9.0, 20.0, 0.5)	130.0 (44.1, 84.2, 1.7)
Snappy	CPU	(23, 128)	(43, 227)	(47, 234)	6.7 (3.6, 2.2, 0.9)	39.8 (15.7, 19.7, 4.4)	168.7 (66.3, 83.4, 19.0)
	GPU	(233, 1036)	(371, 1691)	(387, 1800)	2.6 (0.3, 2.2, 0.1)	21.7 (1.6, 19.7, 0.4)	92.1 (7.2, 83.4, 1.5)
Zstd	CPU	(19, 61)	(31, 89)	(39, 100)	7.5 (4.2, 1.7, 1.6)	44.6 (21.0, 15.2, 8.4)	178.5 (78.4, 65.1, 35.0)
	GPU	(42, 63)	(48, 79)	(48, 79)	4.6 (1.7, 1.7, 1.2)	35.0 (12.3, 15.2, 7.5)	158.2 (57.9, 65.1, 35.2)
Deflate	CPU	(4, 18)	(6, 22)	(5, 20)	24.9 (18.7, 1.8, 4.4)	145.4 (100.9, 15.8, 28.7)	776.5 (563.2, 67.1, 146.2)
	GPU	(37, 107)	(43, 169)	(43, 175)	4.5 (2.0, 1.8, 0.7)	33.1 (13.8, 15.8, 3.5)	147.7 (64.7, 67.1, 15.9)
	SNIC	(46, 85)	(46, 92)	(48, 93)	8.3 (3.6, 1.8, 2.9)	73.2 (32.2, 15.8, 25.2)	307.7 (134.3, 67.1, 106.3)

Table 3 also presents the total communication time between two nodes, along with its breakdown for the compression of intermediate values, the transfer of compressed intermediate values over a 50 Gbps network link, and the decompression of those values. The compression/decompression latency is inversely proportional to the compression/decompression throughput, while the network link latency is proportional to the compression ratio. Zstd and Deflate reduce the network link latency, directly proportional to their compression ratios for gradients and activations (Table 2), although this latency measurement does not capture the reduction in communication time due to the reduced network congestion. Nonetheless, although we choose the platform that can provide the lowest total communication time for Zstd and Deflate (*i.e.*, GPU), we observe that the lossless compression increases it by 95%, 66%, and 74% for Llama-3 70B, GPT-3 175B, and Llama-3 405B, respectively, compared to no compression, due to the long latency associated with compression and decompression for modest compression ratios.

Takeaway-3: Using standard lossless compression algorithms and platforms to compress and decompress intermediate values increases the communication latency, rather than decreasing it, due to the long compression/decompression latency for modest compression ratios at most.

5 NETZIP

In this section, we present NETZIP-algorithm and NETZIP-accelerator. Specifically, NETZIP-algorithm transforms gradients and activations at the bit and value levels for standard lossless compression algorithms to compress them more, exploiting their unique value characteristics. Second, NETZIP-accelerator is a bump-in-the-wire accelerator optimized for achieving high compression/decompression throughput under the hardware resource constraint.

5.1 NETZIP-algorithm

Byte and Bit Grouping for More Dictionary Matching. Figure 5 plots (a) the percentage of 1s at each bit index in gradients and activations in bfloat16 format, and (b) their magnitudes during fine-tuning iterations of Llama-3 70B. In bfloat16 format, bit 15, bits 14–7, and bits 6–0 represent the sign (*S*), exponent (*E*), and mantissa (*M*), respectively. When the exponent satisfies the following condition: $1 \leq E \leq 254$, the numerical value represented by a bfloat16 number is calculated as $(-1)^S \times 2^{E-127} \times (1 + M/2^7)$. The

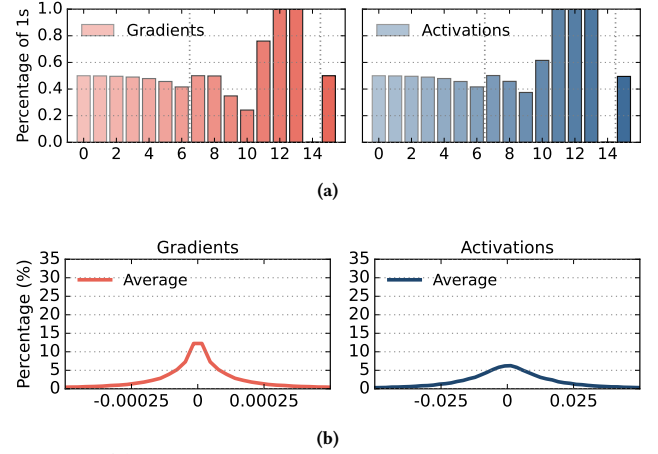


Figure 5: (a) Probability of 1s in each bit of all gradients and activations, and (b) distribution of gradient and activation values of Llama-3 70B.

probability of 1s in the mantissa bits of both gradients and activations is approximately 50%, making them almost incompressible. In contrast, the probability of 1s in the exponent bits of both gradients and activations is high, as a large percentage of them are often close to zero (*e.g.*, $1.07169 \times 10^{-8} = 0_01100100_0111000$), with their

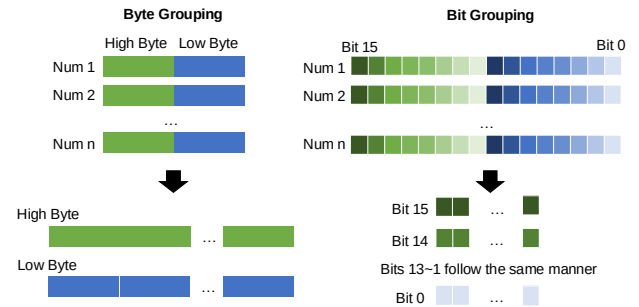


Figure 6: Byte-grouping (left) and bit-grouping (right) for 16-bit bfloat numbers. The same byte and bit positions of different numbers are grouped together.

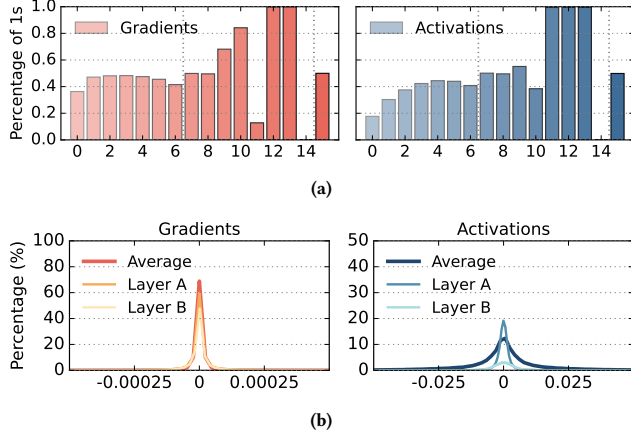


Figure 7: (a) Probability of 1 in each bit of delta values of gradients and activations, and (b) distributions of the delta values over fine-tuning iterations.

magnitudes being both similar and limited in range. Specifically, the exponent bits of gradients are often 011xxxx, where x is either 1 or 0 with a random probability, ranging from 96 to 127. This gives rise to gradients ranging from $2^{96-127} (= 2^{-31})$ to $2^{127-127} (= 2^0)$. The exponent bits of activations are typically 0111xxxx, ranging between 112 and 127, corresponding to activations ranging from 2^{-15} to 2^0 . In other words, the exponent bits have lower entropy than the mantissa bits, making them more compressible.

Based on our observations above, we propose NETZIP-algorithm that first takes only the upper bytes of gradients or activations in bfloat16 format as a byte-group (Figure 6(left)) and compresses it using a standard lossless compression algorithm. The remaining lower bytes form another byte-group, which we may choose not to compress, as they do not compress well and only increase the compression latency. Grouping the upper bytes in such a way can yield more compression of gradients and activations, even with LZ4 and Snappy, which previously failed to compress them. This is because it helps expose more repeated character strings to LZ4 and Snappy for dictionary matching, without being hindered by every other byte containing random values (*i.e.*, the lower bytes) (§2.3). Besides, it is also effective for heavy lossless compression algorithms, such as Zstd and Deflate, which employ both dictionary matching and Huffman coding. Furthermore, we can make the grouping granularity finer, down to taking 1 bit of every gradient or activation with the same bit index, *i.e.*, a total of 16 bit-groups (Figure 6(right)). However, both CPUs and GPUs struggle with such bit-level operations, whereas ASICs or FPGAs excel at them.

Delta Value Compression. Intuitively, especially during fine-tuning, training a given model will incrementally adjust its activations over training iterations. That is, the difference (or delta) of a given gradient or activation value between iterations i and $i + 1$ should be small—often closer to zero than the original value itself. This, in turn, allows a lossless compression algorithm to find more repeated character strings, leading to more compression. Figure 7 plots (a) the percentage of 1s in each bit of delta values of gradients and activations, and (b) the distribution of the delta values over

fine-tuning iterations of Llama-3 70B. To obtain the delta value of each gradient or activation, we subtract its value at the i^{th} training iteration from its value at the $i + 1^{th}$ training iteration. This shows that gradient and activation delta values have more 1s and 0s in their exponent and mantissa, respectively.

Based on the observation above, we propose a delta value compression. Preferably, we store all gradient and activation values from the previous iteration, subtract them from the corresponding values at the current iteration, and then compress these subtracted values. We refer to this as true delta compression. However, doing so is impractical because it will double the memory capacity required to store gradients and activations from both the previous and current iterations. Alternatively, considering that the distributions of gradients and activations are narrow, for example, we choose a single gradient as a base value for the past iteration, and subtract the base value from each gradient value at the current iteration. For instance, by using the minimum activation value as the base value, we can obtain the delta values of activations as follows: $activation_delta[i + 1][j] = activation[i + 1][j] - min(activation[i])$, where i and j represent the iteration number and the activation vector index, respectively. Lastly, as we observe different ranges of gradients and activations across different layers (Figure 7(b)), we use a base value for each layer to get the delta values of gradients and activations of the layer. This eliminates the memory capacity constraint to store all intermediate values from the previous iteration.

5.2 NETZIP-accelerator

Limitations of the SNIC compression accelerator. Using the NVIDIA BlueField-2 SNIC, we have demonstrated that training with compression ends up increasing the inter-node communication time, compared to training without compression, due to a long compression/decompression latency. In addition to the intrinsic compression/decompression latency of the accelerator itself, other factors contribute to the long latency. Specifically, the Deflate accelerator is connected to the SNIC’s embedded PCIe switch. Therefore, for compression, the SNIC’s Arm CPU has to transfer uncompressed intermediate values from its memory to the accelerator’s memory through its internal PCIe interconnect, as is the case with other PCIe-connected accelerators such as a GPU. As the compression accelerator has a limited memory capacity, it can compress only 128 MB of gradients or activations at a time. After compression, the SNIC’s Arm CPU needs to copy the compressed gradients and activations to its memory before it sends them to another node. Our evaluation shows that these transfers contribute to 56% of the compression latency due to the limited memory bandwidth of the SNIC’s Arm CPU. Additionally, if we use NETZIP-algorithm to further decrease the compression ratio, the SNIC’s Arm CPU has to read the gradients and activations from its memory, compute their delta values, byte- or bit-group them, and then send them to the accelerator through its internal PCIe interconnect. As the SNIC’s Arm CPU has less powerful and fewer cores than server-class CPUs [17], it will only further increase the compression latency. We implemented NETZIP on the NVIDIA BlueField-2 SNIC, including data transfer through the SNIC’s Arm CPU and the compression/decompression process. This implementation increased the inter-node communication time by 11× compared to no compression.

The NVIDIA BlueField-3 SNIC supports only a decompression accelerator for the following reasons. The NVIDIA BlueField-2 SNIC implements a Deflate accelerator. Since Deflate is a heavy, resource-intensive lossless compression algorithm, under the hardware resource and power constraints, the Deflate accelerator in the NVIDIA BlueField-2 SNIC can provide a compression throughput of only 50 Gbps, which is only half the SNIC's 100 Gbps line rate. Such a low compression throughput relative to the line rate makes the Deflate accelerator less useful for applications demanding high network bandwidth. Additionally, the Deflate accelerator even at 50 Gbps consumes a significant amount of hardware resources and power (\$4). Given that the NVIDIA BlueField-3 SNIC has doubled the bandwidth of its network link, it may not have the capacity to integrate a Deflate compressor with higher throughput under the hardware resource and power constraints.

Design Space Exploration. To address these limitations, we propose NETZIP-accelerator, a cost-efficient bump-in-the-wire compression accelerator integrated into a NIC ASIC. First, as a bump-in-the-wire accelerator, it eliminates the long latency incurred by transferring gradients and activations among the SNIC's CPU, memory, and accelerator. Second, it implements NETZIP-algorithm and a standard compression algorithm and integrates them within a NIC ASIC. Choosing a standard compression algorithm for NETZIP-accelerator, we consider trade-offs among compression ratio, throughput, latency, hardware resources, and power consumption.

Although heavy algorithms like Zstd and Deflate generally provide higher compression ratios, they present lower throughput and higher latency than lightweight algorithms like LZ4 and Snappy when running on the CPU or GPU. That is, Zstd and Deflate implemented in hardware will also give longer latency, lower throughput, and/or higher power consumption, while requiring more hardware resources than LZ4 and Snappy. Therefore, if NETZIP-algorithm proves effective even for LZ4 and Snappy, NETZIP-accelerator can implement a lightweight compression algorithm with more compressor/decompressor instances than heavy, resource-intensive compression algorithms, delivering higher throughput and lower latency under the hardware resource and power constraints.

Table 4 compares the throughput, latency, and power consumption of FPGA-based LZ4, Snappy, Zstd, and Deflate, where the latency is defined as the time to output the first compressed or decompressed byte for a given stream of input. To synthesize these compression accelerators, we use the Vitis Data Compression Library [5] and map them onto an AMD Alveo U280 Data Center Accelerator Card [4]. Since LZ4 and Snappy provide almost the same throughput, latency, and FPGA resource consumption, we will take LZ4 for the comparison with Zstd and Deflate in this section.

Table 4: Throughput, latency, and power consumption of LZ4, Snappy, Zstd, and Deflate accelerators under the same FPGA resource constraint as LZ4 providing 50 Gbps throughput.

	(Compress, Decompress)			
	LZ4	Snappy	Zstd	Deflate
Throughput (Gbps)	(50, 50)	(50, 50)	(30, 7)	(24, 33)
Latency (μ s)	(2.0, 0.2)	(2.0, 0.2)	(4.1, 1.4)	(4.3, 1.1)
Power (W)	(6, 3)	(6, 3)	(6, 3)	(6, 4)

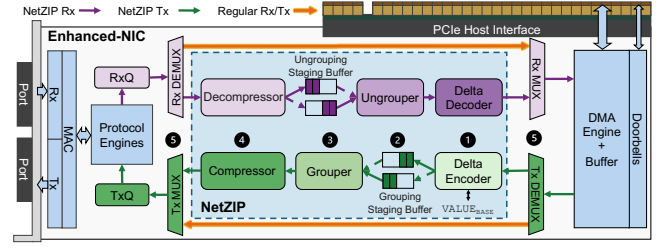


Figure 8: NETZIP-accelerator integrated with a standard NIC.

To achieve 50 Gbps throughput, LZ4 consumes 17% of the FPGA resources. Under the same or similar resource constraints, LZ4 provides $1.7\times$ and $2.1\times$ higher throughput than Zstd and Deflate, respectively, which only achieve 30 Gbps and 24 Gbps throughput. This is because LZ4 consumes only 13% and 20% of the resources that Zstd consumes for a compressor and decompressor instance, respectively. LZ4 also offers 51% and 86% lower compression and decompression latency than Zstd, respectively.

Design. NETZIP-accelerator operates as a bump-in-the-wire accelerator, placed between the DMA Engine+Buffer and the Protocol Engine in a NIC ASIC, as depicted in Figure 8, where we compress gradients and activations per 4 kB packet. Since distributed training frameworks use the RDMA protocol and send a large volume of gradients and activations in a bursty manner, they employ the jumbo packet size (*i.e.*, 4 kB) for all packets to minimize packet header overhead. To selectively compress and decompress only the packet payloads containing gradients or activations, we adapt RDMA verbs to allow the distributed training frameworks to append a custom header—a 1-bit compression flag and a 15-bit layer ID—to each packet. This can support $32,768 (=2^{15})$ layer IDs, while Llama-3 405B has 128 layers.

NETZIP-accelerator comprises compression and decompression paths, as illustrated in Figure 8. The compression path consists of ① Delta Encoder, ② Grouping Staging Buffer, ③ Grouper, ④ Compressor, and ⑤ Tx (DE)MUX. Since not every packet is an RDMA packet storing intermediate values, to prevent unnecessary compression of such a packet, the Tx DEMUX steers the packet to either the Protocol Engine or the Delta Encoder based on the compression flag in the custom header. The Delta Encoder is composed of three units: (1) a base value compute unit, (2) a delta value compute unit, and (3) base value registers. To use the minimum value of gradients or activations in a given layer for the base value, the base value compute unit first stores the maximum possible value in bfloat16 format in a delta value register, treating it as the current minimum value. As it receives packets from the DMA Engine+Buffer through Tx DEMUX, it compares the current minimum value with each bfloat16 value in the payloads of these packets associated with the same layer ID. If the compared value is smaller than the current minimum value in the register, it is stored in the register as the current minimum value. The delta value compute unit gets the delta value for each intermediate value (*i.e.*, gradient or activation) in a packet payload. After computing the delta values of all intermediate values, it passes them, along with the base value, to the Grouping Staging Buffer.

To provide efficient compression and minimum latency, we adopt a dual-buffer architecture for the Grouping Staging Buffer. While one buffer is receiving a 4 kB packet, the other buffer, which stores the preceding packet, is used by the Grouper to perform grouping of delta values in byte or bit granularity before sending them to the Compressor. The Compressor compresses the grouped values and appends the base value to the end of the payload. In case the size of the compressed payload exceeds 4 kB—which has not happened in our evaluation—it simply discards the compressed packet and has the Tx MUX send the uncompressed packet to the Protocol Engine after removing the custom header.

The decompression path is the reverse of the compression path, consisting of Rx DEMUX, Decompressor, Ungrouping Staging Buffer, Ungrouper, Delta Decoder, Rx MUX. When receiving a packet from the Protocol Engine, the Rx DEMUX checks its custom header to determine whether the packet is compressed. If it finds that the packet is uncompressed, it sends the packet directly to the DMA Engine+Buffer through the Rx MUX. Otherwise, it sends the packet to the Decompressor. After decompressing the payload, the Decompressor writes it to one of the buffers in the Ungrouping Staging Buffer. The Ungrouper rearranges the order of bits or bytes to their original order and sends the payload, along with the base value, to the Delta Decoder. Subsequently, the Delta Decoder adds the base value to every delta value, completing the restoration of all intermediate values in the payload. Lastly, it sends the packet to the DMA Engine+Buffer.

ASIC Implementation Analysis. Our FPGA implementation is based on the Xilinx U280 (16nm), currently operating at a clock rate of 300 MHz. With an ASIC implementation using a 7nm process—common in modern NIC ASICs [21], we estimate the design could be optimized to operate at up to 1 GHz. Using the methodology of Kuon and Rose to project FPGA characteristics to ASIC [6, 24], the estimated area is approximately $2mm^2$ (less than 0.1% of the CX-7 NIC physical area [43]), and power consumption is around 0.5 W (about 2% of the CX-7 NIC’s thermal limit of 25 W [43]). The estimated pipeline depth is 170. NETZIP is highly compatible with evolving NIC architectures.

Compatibility with other distributed training frameworks. NETZIP is transparently applicable to today’s leading distributed training frameworks, such as PyTorch [48], DeepSpeed [52], and TensorFlow [1] as NETZIP changes only the network communication layer; no changes are required to the core training code. Specifically, all these frameworks handle their communication calls through a backend such as NCCL [42], which in turn issues RDMA verbs. These frameworks can expose the current layer ID by extending NCCL calls with an additional field, such as `ncclAllReduce()`. Inside NCCL, we just need to append a custom header to each packet before invoking `ibv_post_send()`, the RDMA verb that submits the work request to the NIC. This insertion occurs in `ncclIbIsend()` within NCCL, so everything above that point, including the core training logic, remains unchanged.

6 Evaluation

6.1 Setup

Systems. Table 5 summarizes the configuration of our server used to evaluate the compression ratio, throughput, and latency of LZ4,

Table 5: Evaluation system configurations.

Server	Description
OS	Ubuntu 24.04 LTS (Linux 6.8.0)
Processor Model	Intel Xeon Platinum 8460Y+ (Sapphire Rapids)
# Cores	80 (2 sockets)
System Memory	1 TB DDR5-4800, 16 DIMMs, 16 channels
Devices	Description
NIC	NVIDIA ConnectX-6 NIC NVIDIA BlueField-2 SmartNIC
FPGA	Alveo U280 Data Center Accelerator Card
GPU	NVIDIA H100 Graphics Card, 80GB HBM3

Snappy, Zstd, and Deflate running on the CPU, GPU, and SNIC (Section 4). The server is equipped with two 4th-generation Intel Xeon Scalable Processors with 1 TB of memory, an NVIDIA H100 GPU, and an NVIDIA BlueField-2 SNIC. To evaluate node-to-node communication time, we first set up two servers, each equipped with an NVIDIA ConnectX-6 NIC. Then, we connect NETZIP-accelerator implemented in an AMD Alveo U280 Data Center Accelerator Card to a standard NIC through an Ethernet cable to emulate a NETZIP-accelerator NIC, as shown in Figure 9, since no open-source FPGA-based NIC design for this FPGA device supports the full RDMA stack. The purpose of this implementation is twofold. First, it facilitates the end-to-end functional verification of a NETZIP-accelerator NIC in a full-system setup connecting two commodity servers. Second, it provides latency values for all NETZIP-accelerator components, with each implemented to meet a target throughput of 50 Gbps. We use these latency values to evaluate the communication time between two commodity servers by reducing the packet payload sizes in proportion to the compression ratios of intermediate values achieved by NETZIP-algorithm and adding the latency values before sending the packets from one server to the other. We also use them to evaluate communication time across 512 nodes in a cluster by plugging them into SimAI. We use SimAI because large-scale, end-to-end training demands hundreds or even thousands of GPUs (512 GPUs in our evaluation), and thus it is prohibitively expensive to equip each node with an FPGA device prototyping NETZIP. Lastly, NETZIP-accelerator, including both the compression and decompression paths, consumes 21% of the FPGA device, where the Compressor, the Decompressor, and the remaining components contribute to 17.4%, 0.6%, and 3.0%, respectively.

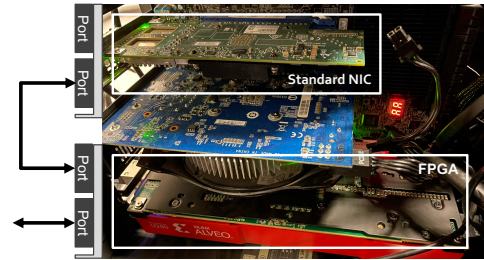


Figure 9: Bump-in-the-wire setup to connect an FPGA-based NETZIP-accelerator to a standard NIC.

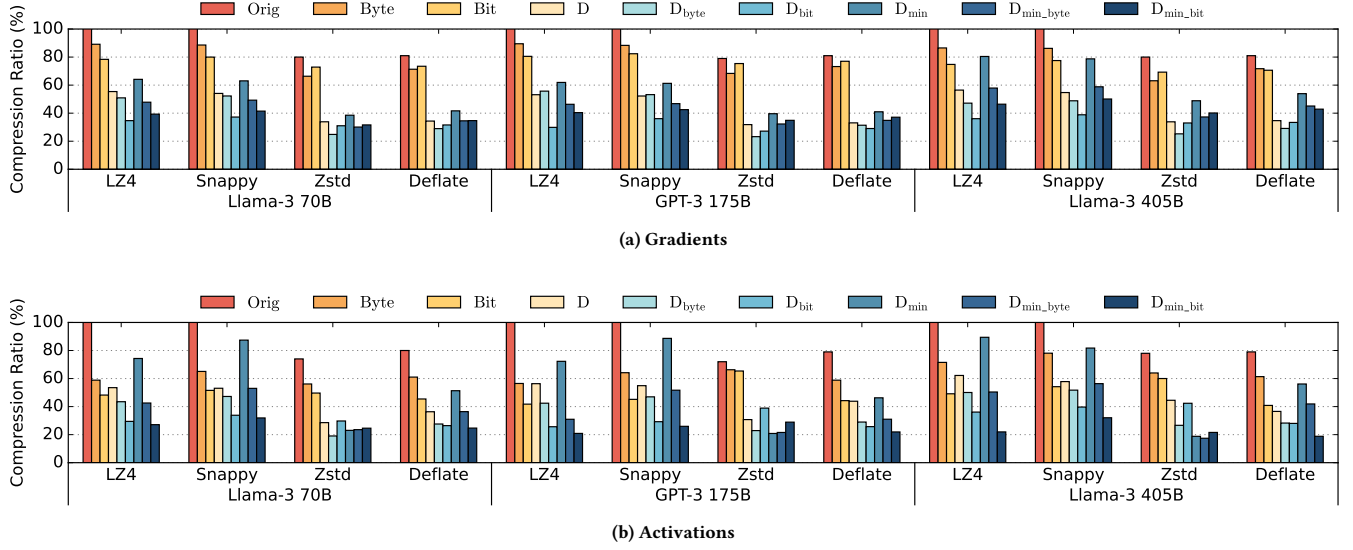


Figure 10: Compression ratios for (a) gradients and (b) activations for the following cases: original (Orig), byte-grouped (Byte), bit-grouped (Bit), and true delta value compression (D), delta value compression with minimum value subtraction ($D_{\min}$), as well as combinations of the previous enhancements (D_{byte} , D_{bit} , $D_{\min_byte}$ and $D_{\min_bit}$).

Collection of gradients and activations. We evaluate three models: Llama-3 70B, GPT-3 175B, and Llama-3 405B. The parameters of Llama-3 70B and Llama-3 405B are obtained from Hugging Face [19]. Since the parameters of GPT-3 175B are not publicly available, we use BLOOM 176B [69], which approximates the scale of GPT-3 175B to collect gradients and activations during fine-tuning. However, for simulating communication time across 512 nodes in a cluster, we use GPT-3 175B since SimAI supports GPT-3 175B but not BLOOM 176B. We fine-tune these models using the Alpaca dataset [58] to compute gradients and activations and collect them.

6.2 Compression Ratios

Figure 10 shows the compression ratios for gradients and activations of Llama-3 70B, GPT-3 175B, and Llama-3 405B, achieved by LZ4, Snappy, Zstd, and Deflate, assisted by NETZIP-algorithm. To distinguish these compression algorithms from standard LZ4, Snappy, Zstd, and Deflate, we refer to them as NETZIP-LZ4, NETZIP-Snappy, NETZIP-Zstd, and NETZIP-Deflate, respectively, henceforth. Since NETZIP-LZ4 provides almost the same throughput, latency, and compression ratios as NETZIP-Snappy in their hardware implementations, we focus on NETZIP-LZ4 in this section. Likewise, as NETZIP-Zstd often provides higher throughput, lower latency, and lower compression ratios than NETZIP-Deflate, we focus on NETZIP-Zstd in this section. Revisiting Table 2, we note that LZ4 could not compress either gradients or activations of all three models at all, while Zstd and Deflate could reduce the volumes of gradients and activations by 19%–28%.

Grouping. When assisted by byte-grouping, NETZIP-LZ4 reduces the volumes of gradients and activations by 12% and 38%, respectively, on average across all three models. Zstd also benefits from byte-grouping, although to a lesser extent than LZ4, especially for activations. On average, NETZIP-Zstd assisted by the byte-grouping

reduces the volumes of gradients and activations by 34% and 38%, respectively, which are 13 and 11 percentage points more than Zstd.

Assisted by bit-grouping, NETZIP-LZ4 reduces the volume of gradients and activations by 22% and 54%, respectively, on average across all three models. These compression ratios are very close to or lower than those of Zstd, capturing the benefit of using the heavy compression algorithms with simple bit-grouping hardware. NETZIP-Zstd, assisted by bit-grouping, reduces the volumes of gradients and activations by 28% and 42%, respectively. Note that bit-grouping for gradients is less effective than byte-grouping, as it increases the compression ratios (volume of gradients) by 7 percentage points, on average across all three models. We speculate that the bit-level manipulation of gradients makes Huffman coding less effective than the byte-level manipulation. Nevertheless, on average, NETZIP-Zstd, assisted by bit-grouping, still reduces the volumes of gradients and activations by 7 and 14 percentage points more than Zstd, respectively.

Delta compression. True delta compression alone is as effective as bit-grouping and byte-grouping for LZ4. NETZIP-LZ4, assisted by delta compression, reduces the volumes of gradients and activations by 45% and 44%, respectively, on average across all three models. In contrast, delta compression alone is notably more effective than byte-grouping for Zstd. NETZIP-Zstd, assisted by delta compression, reduces the volumes of gradients and activations by 67% and 66%, respectively, on average. This reduction is 46 and 30 percentage points more than Zstd, and by 39 and 22 percentage points more than Zstd assisted by bit-grouping, respectively. Furthermore, delta compression has synergies with both byte-grouping and bit-grouping. For example, NETZIP-LZ4, assisted by both delta compression and bit-grouping, reduces the volumes of gradients and activations by 67% and 70%, respectively, which are 21 and 27

percentage points more than delta compression alone, and 44 and 16 percentage points more than bit-grouping.

While highly effective, true delta compression is not feasible, as a standard NIC lacks sufficient memory capacity to store all the intermediate values from the previous iteration. Instead, we have proposed using a single base value for each layer—the minimum intermediate value of a given layer (§5.1). NETZIP-LZ4, assisted by delta compression with a single base value, reduces the volumes of gradients and activations by 12 and 21 percentage points less, respectively, than LZ4 assisted by true delta compression, on average across the three models. Yet, it achieves 32 and 21 percentage points more than LZ4, respectively. NETZIP-Zstd, assisted by delta compression with a single base value, reduces the volume of gradients by 8 percentage points less than NETZIP-Zstd assisted by true delta compression, but still achieves a significant reduction. Lastly, delta compression with a single base value also exhibits synergies with bit-grouping, further reducing the volumes of intermediate values compared to using bit-grouping or delta compression alone. **Effectiveness on other models and precisions.** In this work, we focus on models in the bfloat16 format because the weights of most open-source models are available only in this format [7, 36, 38], and other lower precision formats are unlikely to be used for large model training. Regardless, we report the compression ratios for the gradients and activations of Mixtral 8×7B in bfloat8 format, along with those in bfloat16 format in Figure 11. The compression ratios for the gradients and activations in bfloat16 format are 56% and 50%, respectively, on average, with bit-grouping and delta value compression ('Bit' and 'D_{bit}' in Figure 11). The compression ratios for those in bfloat8 are 49% and 24%, respectively, on average, with bit-grouping and delta value compression.

6.3 Communication Time

Node-to-node communication time. We first evaluate the compression/decompression latency of NETZIP-accelerator, normalized to that of GPU-based Snappy, which gives the lowest compression/decompression latency in Table 3 when one node is sending compressed gradients and activations of Llama-3 70B, GPT-3 175B, and Llama-3 405B to another node. Figure 12 shows that, on average across all three models, NETZIP-LZ4 reduces compression and decompression latency by 756× and 957×, respectively. NETZIP-Zstd reduces compression and decompression latency by 369× and 280×, respectively, but still 2.1× and 7.0× longer than NETZIP-LZ4. This is because NETZIP-Zstd has 1.7× and 7.1× lower compression and

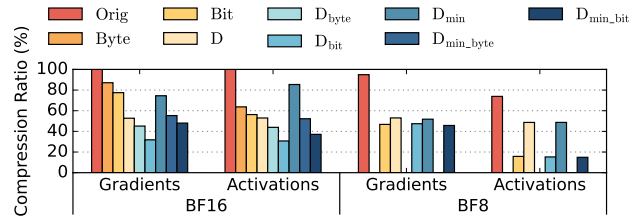


Figure 11: Compression ratios for gradients and activations of Mixtral 8×7B using bfloat16 and bfloat8; byte-grouping is not applicable to bfloat8.

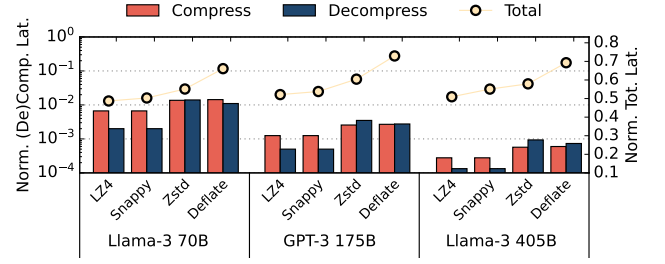


Figure 12: NETZIP-accelerator compression/decompression latency, normalized to the GPU-based Snappy compression/decompression latency, and communication time normalized to the communication time without compression.

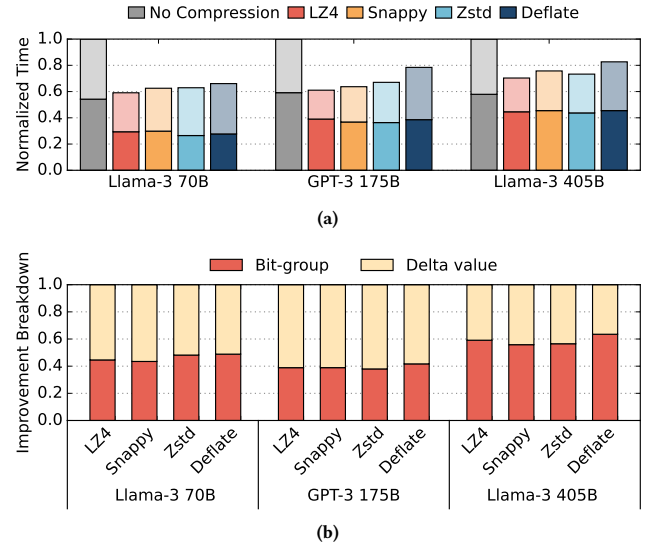


Figure 13: (a) Average per-iteration inter-node communication time of NETZIP, normalized to that of no compression in a cluster of 512 nodes, and the lighter segment at the top of each bar denotes the contributions from network congestion; and (b) breakdown of the communication time reduction between bit-group and delta value compression.

decompression throughput, respectively, than NETZIP-LZ4 under the same FPGA resource constraint.

Next, we evaluate the node-to-node communication time with NETZIP-LZ4, NETZIP-Snappy, NETZIP-Zstd, and NETZIP-Deflate, normalized to the communication time without compression. Figure 12 shows that NETZIP-LZ4 can reduce the average communication for sending and receiving intermediate values by 43%, 43%, and 44% for Llama-3 70B, GPT-3 175B, and Llama-3 405B, respectively, compared to the communication time without compression. Note that, although GPU-based Snappy is the fastest among all compression algorithms running on the CPU, GPU, and SNIC, it still increases the communication by 18%, 10%, and 10% for Llama-3 70B, GPT-3 175B, and Llama-3 405B, respectively, compared to communication time without compression.

Cluster-wide communication time. Figure 13(a) shows the average inter-node communication time across 512 nodes in a cluster, simulated using SimAI. We modified to account for the compression ratios of gradients and activations of Llama-3 70B, GPT-3 175B, and Llama-3 405B from Figure 10, as well as the corresponding compression and decompression latency from Figure 12. This shows that, although NETZIP-LZ4, -Snappy, and -Zstd provide similar reduction in communication time per iteration, NETZIP-LZ4 gives the lowest communication time for all three models, reducing it by 36%, compared to the communication time without compression.

Communication time breakdown. Figure 13(a) also plots the breakdown of inter-node communication time into network congestion and packet transmission components. Without NETZIP, communication accounts for 93–97% of the total training time, with 43% of the communication time attributed to network congestion and the remaining 57% to packet transmission. With NETZIP, communication time is reduced by 30%–41%, and the contribution from network congestion drops to 26%. We further break down the reduction in communication time into contributions from bit-grouping and delta value compression in Figure 13(b). For Llama-3 70B, GPT-3 175B, and Llama-3 405B, bit-group accounts for 39%, 46%, and 58% of the reduction, respectively, while the delta value contributes 61%, 54%, and 41% reduction on average.

Total training time. For Llama-3 70B, GPT-3 175B, and Llama-3 405B, the communication time across 512 nodes in a cluster without compression contributes to 97%, 96%, and 93% of the total training time, respectively. With the reduction in communication time per training iteration achieved by NETZIP, the total training time of these three models is reduced by 40%, 39%, and 28%, respectively.

6.4 Comparison with Lossy Compression

In Figure 14, we compare the training loss of Llama-3 70B during fine-tuning among (1) no compression, (2) NETZIP, and (3) top-K lossy compression. The loss curve of NETZIP does not exhibit any perceivable difference from that of no compression throughout the entire epoch. In contrast, the 75% top-K lossy compression incurs a noticeably higher training loss than NETZIP, and it also results in a 1.4× increase in training time per iteration. After one epoch, both no compression and NETZIP achieve a validation loss of 1.2, whereas the 75% top-K lossy compression incurs a significantly higher validation loss (1.9). In addition, increasing the top-K lossy compression to match the compression ratio attained by NETZIP (36%) causes convergence failure.

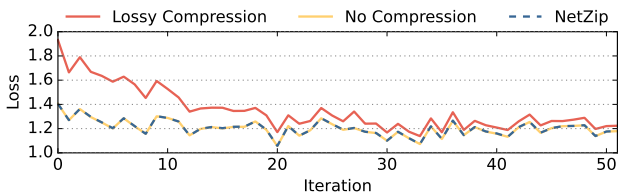


Figure 14: Comparison of training loss across iterations for Llama-3 70B under three settings.

7 Related Work

Compression in model training. In model training, gradient compression has become an important technique for reducing communication bottlenecks. There has been a large body of research focusing on improving communication efficiency in large-scale training, such as using asynchrony [29, 40, 70], decentralization [30, 32], gradient quantization [2, 67], gradient sparsification [62, 66], local steps [31, 56], *etc.*. These methods reduce the volume of gradients that need to be exchanged across compute nodes, reducing network congestion and speeding up the overall training time. Nevertheless, unlike NETZIP, they overlook the compression of activations, which we find contributes to a significant percentage of intermediate values (*i.e.*, gradients and activations) exchanged across compute nodes in distributed large model training. Additionally, adaptive techniques have been introduced that adjust the compression ratio based on real-time network conditions. These methods aim to balance the speed and accuracy of each iteration. This balanced approach to minimizing time-to-accuracy (TTA) has shown significant improvements in training speed in distributed training.

Platforms for compression. Traditional proposals often rely on the host CPU for compressing/decompressing intermediate values, but they are suboptimal. For instance, with state-of-the-art communication libraries like NCCL [42], the transfer of intermediate values typically bypasses the host CPU to minimize PCIe bandwidth consumption, avoiding the overhead of multiple transfers of the intermediate values across the GPU, CPU, and NIC. Nonetheless, the host CPU is already burdened with tasks such as data preprocessing and checkpointing, leaving limited resources for compression and decompression [39]. Even with dedicating all the host CPU cores to compression and decompression, we have demonstrated that the CPU-based compression and decompression still increase the communication time in this work (§4).

Using the GPU for compression also presents challenges. Compression on the GPU can interfere with its primary computational tasks, leading to increased execution times. A DGX H100 instance [44] comprises multiple NVIDIA BlueField SNICs, each integrated a hardware-based compression accelerator. However, an NVIDIA BlueField-2 SNIC offers the compression throughput limited to half of the SNIC’s 100 Gbps line rate, while the NVIDIA BlueField-3 SNIC does not provide any compression accelerators at all. The NVIDIA BlueField-3 SNIC has an Arm CPU with 16 cores. Since they are known to be underutilized, they can be used for compressing intermediate values, reducing the compute burden of compression on the host CPU and GPU. Nonetheless, its CPU is far less powerful than typical server-class CPUs [17], and this work has demonstrated that even a server-class CPU struggles with providing high compression throughput at low latency. In contrast, NETZIP explores the use of NICs with bump-in-the-wire accelerators for compressing and decompressing intermediate values.

Efficient model training. Recent advances have enabled large-scale model training across thousands of GPUs—for example, LLaMA-3 requires 16K GPUs—by combining tensor, pipeline, and data parallelism [59, 72]. This degree of parallelism significantly amplifies inter-node communication time, which our work aims to address. To reduce the training cost associated with the inter-node communication time, distributed training frameworks like

Egeria [64] skip updates by adaptively freezing layers, while ModelKeeper [25] bootstraps new models by transferring pretrained weights. SYNDICATE [35] breaks large communication work into smaller pieces as part of execution planning, thereby packing and ordering communication work so as to maximize both network utilization and overlap with compute. Across jobs, CASSINI [51] minimizes network contention by scheduling communication phases (e.g., forward and backward passes) to reduce interference.

Lossless compression of model parameters. Previous work has explored the lossless compression of parameters in small CNN models used for edge inference applications. The lane-compression method [23] groups bit positions with varying entropy levels into multiple lanes and applies the DPRed (Dynamic Precision Reduction) algorithm along with two customized variants—Sparse-DPRed and Dense-DPRed—tailored to compress sparse and dense lanes, respectively, for more compression. The exponent-sharing method [22] reduces redundant exponents by constructing a lookup table containing all unique exponents from a set of floating-point values, and assigns each value to an index to retrieve its corresponding exponent at runtime. NETZIP differs from them in several aspects. First, while they compress parameters of small CNN models, NETZIP compresses *gradients* and *activations* in very large models, which are multiple orders of magnitude larger than those small CNN models, after comprehensively analyzing their compressibility. Second, NETZIP not only rearranges intermediate values at the bit and byte levels but also transforms them using delta compression. Lastly, NETZIP seamlessly works with a standard lossless compression IP integrated with a NIC for in-network compression.

Fully Shared Data Parallelism (FSDP). Recent advances in distributed model training leverage FSDP to reduce the required GPU memory capacity. Specifically, FSDP replaces all-reduce with reduce-scatter and all-gather collectives, reducing network congestion, but it further increases per-iteration network traffic, as it involves distributing and retrieving model parameters, optimizer states, and/or gradients across nodes [12]. This exacerbates the network bandwidth bottleneck, further reinforcing the importance of NETZIP which can significantly reduce network traffic and be transparently applicable to any training framework. However, we cannot quantitatively evaluate NETZIP's effectiveness for distributed training frameworks deploying FSDP as our evaluation framework—SimAI—uses a version of Megatron-LM that does not support FSDP.

8 Conclusion

In this work, we have proposed NETZIP-algorithm, which allows standard lossless compression algorithms to further reduce the volumes of both gradients and activations. Second, we have proposed NETZIP-accelerator, which integrates NETZIP-algorithm with an accelerator for a standard lossless compression algorithm in a bump-in-the-wire fashion to maximize the throughput and latency for compression and throughput under the hardware resource constraints imposed by NIC ASICs. Using our implementation of NETZIP with an FPGA-based NIC, we show that LZ4 or Snappy is the best choice for implementing NETZIP-accelerator among accelerators implementing the standard lossless compression algorithms. Our evaluation shows that NETZIP reduces the total training time by 35%.

Acknowledgments

This work was supported in part by grants from the IBM-Illinois Discovery Accelerator Institute (IIDAI), and MSIT, Korea, under the Global Scholars Invitation Program (RS-2024-00456287) supervised by the IITP; and by a generous gift from AMD-Xilinx HACC. Nam Sung Kim is the corresponding author.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mane, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viegas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. *arXiv preprint arXiv:1603.04467* (2016).
- [2] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. *Advances in neural information processing systems* 30 (2017).
- [3] Amazon. accessed in 2025. Amazon EC2 Instance Types. <https://docs.aws.amazon.com/ec2/latest/instancetypes/ec2-types.pdf>.
- [4] AMD. accessed in 2025. Alveo U280 Data Center Accelerator Card. <https://www.xilinx.com/publications/product-briefs/alveo-u280-product-brief.pdf>.
- [5] AMD. accessed in 2025. Vitis Data Compression Library. https://github.com/Xilinx/Vitis_Libraries/tree/master/data_compression.
- [6] Anton Shilov. accessed in 2025. GlobalFoundries Details 7 nm Plans: Three Generations, 700 mm2, HVM in 2018. <https://www.anandtech.com/show/11558/globalfoundries-details-7-nm-plans-three-generations-700-mm-hvm-in-2018>.
- [7] BigScience Workshop. accessed in 2025. bloom. <https://huggingface.co/bigscience/bloom>.
- [8] CloudLab. accessed in 2025. CloudLab Portal Hardware. <https://www.cloudlab.us/portal-hardware.php>.
- [9] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. 2019. The Design and Operation of CloudLab. In *USENIX annual technical conference (USENIX ATC)*, 1–14.
- [10] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is Not All You Need. *arXiv preprint arXiv:2407.09141* (2024).
- [11] Facebook. accessed in 2025. zstd. <https://github.com/facebook/zstd>.
- [12] Jared Fernandez, Luca Wehrstedt, Leonid Shamis, Mostafa Elhoushi, Kalyan Saladi, Yonatan Bisk, Emma Strubell, and Jacob Kahn. 2024. Hardware Scaling Trends and Diminishing Returns in Large-Scale Distributed Training. *arXiv preprint arXiv:2411.13055* (2024).
- [13] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Rifati, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuaiqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed AI Training at Meta Scale. In *Proceedings of the ACM SIGCOMM Conference (SIGCOMM)*, 57–70.
- [14] Google. accessed in 2025. snappy. <https://github.com/google/snappy>.
- [15] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth

- Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yearly, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsim-poukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shao-liang Nie, Sharan Narang, Sharath Rapparth, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Kan, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gouget, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zachary Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Gupta, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymier, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriella Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Hafeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhee, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Juyert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Prithvi Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangarabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiao-cheng Tang, Xiao-jian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [16] Yutong He, Pengrui Li, Yipeng Hu, Chuyan Chen, and Kun Yuan. 2024. Subspace Optimization for Large Language Models with Convergence Guarantees. *arXiv preprint arXiv:2410.11289* (2024).
- [17] Jinghan Huang, Jiaqi Lou, Srikanth Vanavasam, Xinhao Kong, Houxian Ji, Ipoom Jeong, Danyang Zhuo, Eun Kyung Lee, and Nam Sung Kim. 2024. HAL: Hardware-assisted Load Balancing for Energy-efficient SNIC-Host Cooperative Computing. In *ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. 613–627.
- [18] David A Huffman. 1952. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101.
- [19] Hugging Face. accessed in 2025. Hugging Face. <https://huggingface.co>.
- [20] Ajay Jaiswal, Zhe Gan, Xianzhi Du, Bowen Zhang, Zhangyang Wang, and Yinfei Yang. 2023. Compressing LLMs: The Truth is Rarely Pure and Never Simple. *arXiv preprint arXiv:2310.01382* (2023).
- [21] Justin Pietsch. accessed in 2025. A summary of High Speed Ethernet ASICs. <https://elegantnetwork.github.io/posts/A-Summary-of-Network-ASICs/>.
- [22] Prachi Kashikar, Olivier Sentieys, and Sharad Sinha. 2023. Lossless Neural Network Model Compression Through Exponent Sharing. *IEEE Transactions on Very Large Scale Integration Systems (VLSI)* (2023).
- [23] Yousun Ko, Alex Chadwick, Daniel Bates, and Robert Mullins. 2021. Lane Compression: A Lightweight Lossless Compression Method for Machine Learning on Embedded Systems. *ACM Transactions on Embedded Computing Systems (TECS)* 20, 2 (2021), 1–26.
- [24] Ian Kuo and Jonathan Rose. 2010. *Quantifying and Exploring the Gap Between FPGAs and ASICs*. Springer Science & Business Media.
- [25] Fan Lai, Yinwei Dai, Harsha V. Madhyastha, and Mosharaf Chowdhury. 2023. ModelKeeper: Accelerating DNN Training via Automated Training Warmup. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 769–785.
- [26] Fan Lai, Jie You, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. 2020. Sol: Fast Distributed Computation Over Slow Networks. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 273–288.
- [27] Wenxue Li, Xiangzhou Liu, Yuxuan Li, Yilun Jin, Han Tian, Zhizhen Zhong, Guyue Liu, Ying Zhang, and Kai Chen. 2024. Understanding Communication Characteristics of Distributed Training. In *Proceedings of the Asia-Pacific Workshop on Networking (APNet)*. 1–8.
- [28] Youjie Li, Jongse Park, Mohammad Alian, Yifan Yuan, Zheng Qu, Peitian Pan, Ren Wang, Alexander Schwing, Hadi Esmaeilzadeh, and Nam Sung Kim. 2018. A Network-Centric Hardware/Algorithm Co-Design to Accelerate Distributed Training of Deep Neural Networks. In *Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 175–188.
- [29] Xiangru Lian, Yijun Huang, Yuncheng Li, and Ji Liu. 2015. Asynchronous Parallel Stochastic Gradient for Nonconvex Optimization. *Advances in Neural Information Processing Systems* 28 (2015), 2737–2745.
- [30] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. 2017. Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent. *Advances in neural information processing systems* 30 (2017).
- [31] Tao Lin, Sebastian U Stich, Kumar Kshitij Patel, and Martin Jaggi. 2018. Don't Use Large Mini-Batches, Use Local SGD. *arXiv preprint arXiv:1808.07217* (2018).
- [32] Yucheng Lu and Christopher De Sa. 2021. Optimal Complexity in Decentralized Training. In *International Conference on Machine Learning*. PMLR, 7111–7123.
- [33] lz4. accessed in 2025. lz4. <https://github.com/lz4/lz4>.
- [34] madler. accessed in 2025. zlib. <https://github.com/madler/zlib>.
- [35] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better Together: Jointly Optimizing ML Collective Scheduling and Execution Planning using SYNDICATE. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 809–824.
- [36] Meta Llama. accessed in 2025. Llama-3.1-405B. <https://huggingface.co/meta-llama/Llama-3.1-405B>.
- [37] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large Language Models: A Survey. *arXiv preprint arXiv:2402.06196* (2024).
- [38] Mistral AI. accessed in 2025. Mixtral-8x7B. <https://huggingface.co/mistralai>.
- [39] Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. 2020. Analyzing and Mitigating Data Stalls in DNN Training. *arXiv preprint*

- arXiv:2007.06775 (2020).
- [40] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. 2011. Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. *arXiv preprint arXiv:1106.5730* (2011).
- [41] NVIDIA. accessed in 2025. DOCA SDK Document. <https://docs.nvidia.com/doca/sdk/index.html>.
- [42] NVIDIA. accessed in 2025. NVIDIA Collective Communications Library (NCCL). <https://developer.nvidia.com/nccl>.
- [43] NVIDIA. accessed in 2025. NVIDIA ConnectX-7 Adapter Cards User Manual. <https://docs.nvidia.com/networking/display/connectx7vpi/specifications>.
- [44] NVIDIA. accessed in 2025. NVIDIA DGX H100. <https://openzeka.com/en/wp-content/uploads/2022/04/ai-for-enterprise-dgx-h100-datasheet-nvidia-a4-2146027-r3-web.pdf>.
- [45] NVIDIA. accessed in 2025. NVIDIA nvCOMP. <https://developer.nvidia.com/nvcomp>.
- [46] NVIDIA. accessed in 2025. NVIDIA Spectrum-X Network Platform Architecture. <https://resources.nvidia.com/en-us-accelerated-networking-resource-library/nvidia-spectrum-x>.
- [47] NVIDIA. accessed in 2025. NVLink and NVLink Switch. <https://www.nvidia.com/en-us/data-center/nvlink/>.
- [48] PyTorch. accessed in 2025. PyTorch. <https://pytorch.org>.
- [49] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM Conference (SIGCOMM)*. 691–706.
- [50] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving Language Understanding by Generative Pre-Training. (2018).
- [51] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. CASSINI: Network-Aware Job Scheduling in Machine Learning Clusters. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 1403–1420.
- [52] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (SIGKDD)*. 3505–3506.
- [53] Mikhail I Rudakov, Aleksandr Nikolaevich Beznosikov, Ya A Kholodov, and Alexander Vladimirovich Gasnikov. 2023. Activations and Gradients Compression for Model-Parallel Training. In *Doklady Mathematics*, Vol. 108. Springer, 272–281.
- [54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [55] Jaeyong Song, Jinkyu Yim, Jaewon Jung, Hongsun Jang, Hyung-Jin Kim, Youngsok Kim, and Jinho Lee. 2023. Optimus-CC: Efficient Large NLP Model Training with 3D Parallelism Aware Communication Compression. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 560–573.
- [56] Sebastian U Stich. 2018. Local SGD Converges Fast and Communicates Little. *arXiv preprint arXiv:1805.09767* (2018).
- [57] Hanlin Tang, Shaoduo Gan, Ce Zhang, Tong Zhang, and Ji Liu. 2018. Communication Compression for Decentralized Training. *Advances in Neural Information Processing Systems* 31 (2018).
- [58] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A Strong, Replicable Instruction-Following Model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html> 3, 6 (2023), 7.
- [59] Taegeon Um, Byungsoo Oh, Minyoung Kang, Woo-Yeon Lee, Goeun Kim, Dongseob Kim, Youngtaek Kim, Mohd Muzzammil, and Myeongjae Jeon. 2024. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs. In *USENIX Annual Technical Conference (USENIX ATC)*. 563–578.
- [60] Chengyu Wang, Yuanhao Yue, Jun Huang, and Peng Wang. 2025. Building a Family of Data Augmentation Models for Low-cost LLM Fine-tuning on the Cloud. In *Proceedings of the International Conference on Computational Linguistics: Industry Track (COLING)*. 431–444.
- [61] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).
- [62] Hongyi Wang, Scott Sievert, Zachary Charles, Shengchao Liu, Stephen Wright, and Dimitris Papailiopoulos. 2018. Atomo: Communication-efficient learning via atomic sparsification. *arXiv preprint arXiv:1806.04090* (2018).
- [63] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, Yang Liu, Pengcheng Zhang, Kun Qian, Kunling He, Jiaqi Gao, Ennan Zhai, Dennis Cai, and Binzhang Fu. 2025. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [64] Yiding Wang, Decang Sun, Kai Chen, Fan Lai, and Mosharaf Chowdhury. 2023. Egeria: Efficient DNN Training with Knowledge-Guided Layer Freezing. In *Proceedings of the European conference on computer systems (EuroSys)*. 851–866.
- [65] Zeqin Wang, Ming Wen, Yuedong Xu, Yipeng Zhou, Jessie Hui Wang, and Liang Zhang. 2023. Communication compression techniques in distributed deep learning: A survey. *Journal of Systems Architecture* 142 (2023), 102927.
- [66] Jianqiao Wangni, Jialei Wang, Ji Liu, and Tong Zhang. 2017. Gradient Sparsification for Communication-Efficient Distributed Optimization. *arXiv preprint arXiv:1710.09854* (2017).
- [67] Wei Wen, Cong Xu, Feng Yan, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. 2017. TernGrad: Ternary Gradients to Reduce Communication in Distributed Deep Learning. *arXiv preprint arXiv:1705.07878* (2017).
- [68] Wikipedia. accessed in 2025. LZ77 and LZ78. https://en.wikipedia.org/wiki/LZ77_and_LZ78.
- [69] BigScience Workshop, :, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, Dragomir Radev, Eduardo González Ponferrada, Efrat Levkovich, Ethan Kim, Eyal Bar Natan, Francesco De Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady Elsahar, Hamza Benaymina, Hieu Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier de la Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jörg Froehberg, Joseph Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro Von Werra, Leon Weber, Long Phan, Loubna Ben allal, Ludovic Tanguy, Manan Dey, Manuel Romero Muñoz, Maraim Masoud, Maria Grandury, Mario Šaško, Max Huang, Maximin Coavoux, Mayank Singh, Mike Tian-Jian Jiang, Minh Chien Vu, Mohammad A. Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaila Khamis, Olivier Nguyen, Omar Espejel, Ona de Gibert, Paula Villegas, Peter Hendersson, Pierre Colombo, Priscilla Amuok, Quentin Lhoest, Rhea Harlman, Rishi Bommasani, Roberto Luis López, Rui Ribeiro, Salomey Osei, Sampo Pyysalo, Sebastian Nagel, Shamik Bose, Shamsuddeen Hassan Muhammad, Shanya Sharma, Shayne Longpre, Somaieh Nikpoor, Stanislav Silberberg, Suhas Pai, Sydney Zink, Tiago Timponi Torrent, Timo Schick, Tristan Thrush, Valentin Danchev, Vassilina Nikoulina, Veronika Laipala, Violette Lepercq, Vrinda Prabhu, Zaid Alyafeai, Zeerak Talat, Arun Raja, Benjamin Heinzerling, Chenglei Si, Davut Emre Taşar, Elizabeth Salesky, Sabrina J. Mielke, Wilson Y. Lee, Abheesh Sharma, Andrea Santilli, Antoine Chaffin, Arnaud Stiegler, Debajyoti Datta, Eliza Szczecchia, Gunjan Chhablani, Han Wang, Harshit Pandey, Hendrik Strobelt, Jason Alan Fries, Jos Rozen, Leo Gao, Lintang Sutawika, M Saiful Bari, Maged S. Al-shaibani, Matteo Manica, Nihal Nayak, Ryan Teehan, Samuel Albanie, Sheng Shen, Srulik Ben-David, Stephen H. Bach, Taewoon Kim, Tali Bers, Thibault Fevry, Trishala Neeraj, Urmish Thakker, Vikas Raunak, Xiangru Tang, Zheng-Xin Yong, Zhiqing Sun, Shaked Brody, Yallow Uri, Hadar Tojari, Adam Roberts, Hyung Won Chung, Jaesung Tae, Jason Phang, Ofir Press, Conglong Li, Deepak Narayanan, Hatim Bourfoune, Jared Casper, Jeff Rasley, Max Ryabinin, Mayank Mishra, Minjia Zhang, Mohammad Shoeybi, Myriam Peyrounette, Nicolas Patry, Noumane Tazi, Omar Sanseviero, Patrick von Platen, Pierre Cornette, Pierre François Lavallée, Rémi Lacroix, Samyam Rajbhandari, Sanchit Gandhi, Shaden Smith, Stéphane Requena, Suraj Patil, Tim Dettmers, Ahmed Baruwaa, Amanpreet Singh, Anastasia Cheveleva, Anne-Laure Ligozat, Arjun Subramonian, Aurélie Névél, Charles Lovering, Dan Garrette, Deepak Tunuguntla, Ehud Reiter, Ekaterina Taktasheva, Ekaterina Voloshina, Eli Bogdanov, Genta Indra Winata, Hailey Schoelkopf, Jan-Christoph Kalo, Jekaterina Novikova, Jessica Zosa Forde, Jordan Clive, Junjo Kasai, Ken Kawamura, Liam Hazan, Marine Carpuat, Miruna Clinciu, Najoung Kim, Newton Cheng, Oleg Serikov, Omer Antverg, Oskar van der Wal, Rui Zhang, Ruochen Zhang, Sebastian Gehrmann, Shachar Mirkin, Shani Pais, Tatiana Shavrina, Thomas Scialom, Tian Yun, Tomasz Limisiewicz, Verena Rieser, Vitaly Protasov, Vladislav Mikhailov, Yada Praksachatkun, Yonatan Belinkov, Zachary Bamberger, Zdeněk Kasner, Alice Rueda, Amanda Pestana, Amir Feizpour, Ammar Khan, Amy Faranak, Ana Santos, Anthony Hevia, Antigona Unldreaj, Arash Aghagol, Arezoo Abdollahi, Aycha Tammour, Azadeh HajiHosseini, Bahareh Behroozi, Benjamin Ajibade, Bharat Saxena, Carlos Muñoz Ferrandis, Daniel McDuff, Danish Contractor, David Lansky, Davis David, Douwe Kiela, Duong A. Nguyen, Edward Tan, Emi Baylor, Ezinwanne Ozoani, Fatima Mirza, Frankie Ononwu, Habib Rezaeejad, HESSIE Jones, Indrani Bhattacharya, Irene Solaiman, Irina Sedenko, Isar Nejadgholi, Jesse Passmore, Josh Seltzer, Julio Bonis Sanz, Livia Dutra, Mairon Samagaio, Maraim Elbadri, Margot Mieskes, Marissa Gerchick, Martha Akinlolu, Michael McKenna, Mike Qiu, Muhammed Ghauri, Mykola Burynok, Nafis Abrar, Nazneen

Rajani, Nour Elkott, Nour Fahmy, Olanrewaju Samuel, Ran An, Rasmus Kromann, Ryan Hao, Samira Alizadeh, Sarmad Shubber, Silas Wang, Sourav Roy, Sylvain Viguier, Thanh Le, Tobi Oyeade, Trieu Le, Yoyo Yang, Zach Nguyen, Abhinav Ramesh Kashyap, Alfredo Palasciano, Alison Callahan, Anima Shukla, Antonio Miranda-Escalada, Ayush Singh, Benjamin Beilharz, Bo Wang, Caio Brito, Chenxi Zhou, Chirag Jain, Chuxin Xu, Clémentine Fourier, Daniel León Perinán, Daniel Molano, Dian Yu, Enrique Manjavacas, Fabio Barth, Florian Fuhrmann, Gabriel Altay, Giyaseddin Bayrak, Gully Burns, Helena U. Vrabec, Imane Bello, Ishani Dash, Jihyun Kang, John Giorgi, Jonas Golde, Jose David Posada, Karthik Rangasai Sivaraman, Lokesh Bulchandani, Lu Liu, Luisa Shinzato, Madeleine Hahn de Bykhovetz, Maiko Takeuchi, Marc Pàmies, Maria A Castillo, Marianna Nezhurina, Mario Sängner, Matthias Samwald, Michael Cullan, Michael Weinberg, Michiel De Wolf, Mina Mihaljcic, Minna Liu, Moritz Freidank, Myungsun Kang, Natasha Seelam, Nathan Dahlberg, Nicholas Michio Broad, Nikolaus Muellner, Pascalle Fung, Patrick Haller, Ramya Chandrasekhar, Renata Eisenberg, Robert Martin, Rodrigo Canalli, Rosaline Su, Ruisi Su, Samuel Cahyawijaya, Samuele Garda, Shlok S Deshmukh, Shubhanshu Mishra, Sid Kiblawi, Simon Ott, Sinee Sang-aroonisiri, Srishti Kumar, Stefan Schweter, Sushil Bharati, Tanmay Laud, Théo Gigant, Tomoya Kainuma, Wojciech Kusa, Yanis Labrak, Yash Shailesh Bajaj, Yash Venkatraman, Yifan Xu, Yingxin Xu, Yu Xu, Zhe Tan, Zhongli Xie, Zifan Ye, Mathilde Bras, Younes Belkada, and Thomas Wolf. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv:2211.05100 [cs.CL]* <https://arxiv.org/abs/2211.05100>

- [70] Cong Xie, Sanmi Koyejo, and Indranil Gupta. 2020. Zeno++: Robust Fully Asynchronous SGD. In *International Conference on Machine Learning (ICML)*. 10495–10503.
- [71] Zhichao Xu, Ashim Gupta, Tao Li, Oliver Benthani, and Vivek Srikumar. 2024. Beyond Perplexity: Multi-dimensional Safety Evaluation of LLM Compression. *arXiv preprint arXiv:2407.04965* (2024).
- [72] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 559–578.

A Artifact Appendix

A.1 Abstract

This section explains how to run experiments for the paper’s key results. We organize the steps into three parts: (1) collect intermediate values (gradients/activations) during fine-tuning to support other experiments; (2) evaluate compression ratios for NETZIP-algorithm (byte/bit grouping and delta-value compression) with LZ4/Snappy/Zstd/Deflate, as shown in Figure 10; and (3) run the SimAI large-scale experiments to quantify end-to-end impact, corresponding to Figure 3, 4, and 13.

A.2 Artifact check-list (meta-information)

- **Algorithm:** LZ4 [33], Snappy [14], Zstd [11] and Deflate [34].
- **Program:** SimAI [63].
- **Model:** Llama-3 70B, GPT-3 175B, Llama-3 405B.
- **Run-time environment:** Docker (NVIDIA PyTorch container), Transformers, Datasets, NumPy; compression libraries of LZ4, Snappy, Zstd, and Deflate.
- **Metrics:** Compression ratio, compression/decompression throughput and latency, and end-to-end training/communication time.
- **Output:** Text/CSV-like results placed under user-specified folders.
- **Experiments:** (1) Data collection; (2) compression ratio analysis with NETZIP-algorithm; (3) SimAI-based large-scale runs.
- **How much disk space required (approximately)?**: 50 GB for functional test.
- **How much time is needed to complete experiments (approximately)?**: 2 hours for functional test.
- **Publicly available?**: Yes.
- **Code licenses (if publicly available)?**: MIT License.
- **Archived (provide DOI)?**: 10.5281/zenodo.16976212

A.3 Description

A.3.1 How to access. Clone the repository and enter the project directory:

```
git clone https://github.com/ece-fast-lab/MICRO-2025-NetZIP.git
cd MICRO-2025-NetZIP
```

A.3.2 Hardware dependencies. The evaluation platform consists of a CPU server equipped with dual Intel Xeon Sapphire Rapids processors, 1 TB of DDR5 memory, NVIDIA H100 GPU, and Mellanox ConnectX-6 (CX-6) NIC. An NVIDIA DGX platform (8×H100 GPUs) may be used to accelerate experiments.

A.3.3 Software dependencies. Containerized environments based on the NVIDIA PyTorch Docker image are used. The software stack includes the Hugging Face Transformers and Datasets libraries for model and data management, as well as NumPy for numerical operations. To evaluate compression schemes, libraries for LZ4, Snappy, Zstd, and Deflate are used. SimAI is employed for large-scale simulation of multi-GPU environments.

A.4 Installation

To run experiments, follow the steps below to set up the required environment using Docker and Python packages.

Docker. We use NVIDIA’s official PyTorch container. Mount the current directory and launch the container:

```
./start_docker.sh
cd /workspace/NetZIP
```

Environment. Once inside the container, install the necessary Python packages, including libraries for model training, dataset handling, numerical operations, and compression:

```
pip install torch transformers datasets huggingface_hub numpy lz4
snappy zstandard tqdm
huggingface-cli login
```

A.5 Experiment workflow

We use the smaller model to do functional tests in this part.

1) Data collection. We collect intermediate training data (e.g., gradients or activations) from a pretrained language model:

```
cd data_collection
./run1.sh
```

The `model_name` specifies the Hugging Face model identifier to load. The `num_steps` sets the number of iterations to collect the data.

2) Compression analysis with NETZIP-algorithm. Evaluate compression ratios with saved intermediate data:

```
cd compression_ratio_calculation
./run2.sh
```

The `dir_step0` and `dir_step1` flags specify the input directories for data from two consecutive training iterations. The `output_dir`

sets the destination directory for results, and `num_workers` defines the number of parallel threads to accelerate computation.

3) Large-scale simulation. Run SimAI to emulate large-scale training with varying bandwidths and model sizes.

```
cd large_scale_simulation
./get_SimAI.sh
cd script
./run3.sh
```

The first argument sets the model size. The second argument specifies the network topology. The third argument is the global batch size . The remaining values are bandwidth settings in Gbps.

A.6 Evaluation and expected results

LZ4, for instance, enhanced by NETZIP-algorithm compress gradients and activations 60 and 70 percentage points more than standard LZ4, respectively. End-to-end training with NETZIP provides 35% lower training time than training without compression.